

Faculty of Engineering

Topics in Computer Science Agents

PhD. Willy Ugarte Rojas
willyugarte@gmail.com



UPC

Universidad Peruana
de ciencias Aplicadas

Question

How would you get to Mountain View on Friday night in the least amount of time?

bike

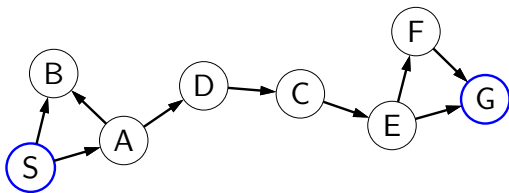
drive

Caltrain

Uber/Lyft

fly

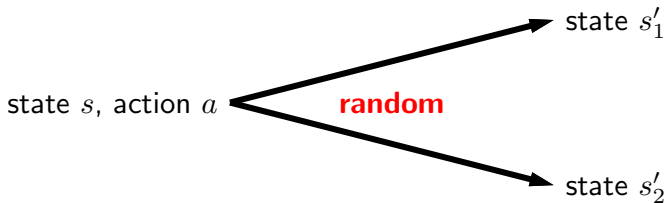
So far: search problems



state s , action a **deterministic** $\longrightarrow$ state $\text{Succ}(s, a)$



Uncertainty in the real world



Volcano crossing



(or press ctrl-enter)

		-50	20
		-50	
2			

Dice game



Example: dice game

For each round $r = 1, 2, \dots$

- You choose **stay** or **quit**.
- If **quit**, you get \$10 and we end the game.
- If **stay**, you get \$4 and then I roll a 6-sided dice.
 - If the dice results in 1 or 2, we end the game.
 - Otherwise, continue to the next round.

Start

Stay

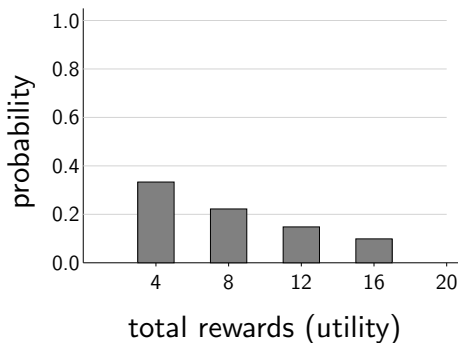
Quit

Dice:

Rewards:

Rewards

If follow policy "stay" :



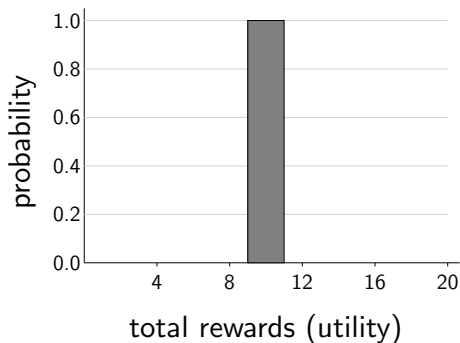
Expected utility:

$$\frac{1}{3}(4) + \frac{2}{3} \cdot \frac{1}{3}(8) + \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3}(12) + \dots = 12$$

- Let's suppose you always stay. Note that each outcome of the game will result in a different sequence of rewards, resulting in a **utility**, which is in this case just the sum of the rewards.
- We are interested in the **expected** utility, which you can compute to be 12.

Rewards

If follow policy "quit" :



Expected utility:

$$1(10) = 10$$

- If you quit, then you'll get a reward of 10 deterministically. Therefore, in expectation, the "stay" strategy is preferred, even though sometimes you'll get less than 10.

Artificial Intelligence

AI and Decision Making

What is behaviour?

The reaction of a subject or an object to its environment?

- passive: a stone falls to the ground
- active: a mosquito flies into the light bulb

What makes behaviour intelligent?

- act like a human
- pursue a certain goal

=> Decide on an action to improve on a goal.

Prediction and Decision Making

How to make decisions to reach a goal?

- What do you know about the current situation ?
- What are your options ?
- Which option is the best?

Problems:

- Parts of your current situation might be unknown or not modeled
- Considering all options is often not possible
- Considering all possible impacts of choosing an option is often not possible.
- Your goal might not be reachable by one decision

Why is this connected to Machine learning?

Uncertain impacts :

- We do not really know what happens if we do something
- How does the environment react?
- How well do we perform the selected action?

Uncertain costs and rewards:

- We do not know if and when an action pays off.
- We cannot exactly predict uncertain factors like waiting times, prices, joy, congestions,...

Uncertain situation:

- We only got some input about our situation.
e.g. sensor data, player view,...
- We might misinterpret a situation.

Applications Sequential Decision Making

- AI for Games:
 - Competitive agents: Alpha Go, Atari Games, Chess,...
 - Realistic opponents: football simulations,
- Controlling Robots and Systems
 - Autonomous Cars
 - Drones
 - Industrial Robots
 - Power stations
- Communicate with people
 - Chatbots for Technical Support
 - Ordering Assistants (e.g. reservations)
 - Internet advertisement
- Build intelligent assistant systems
 - planning trips and routs
 - optimize processes
 - manage financial portfolio

Reinforcement Learning for board games

- games are classical domain of AI because opponent behaves not just nondeterministic but antagonistic
- progress on games like chess, backgammon, Poker.
- for a long time Go was considered the biggest challenge due to its enormous branching factor
- In Jan 2016: Deepmind challenged Grand Master Lee Sedol with a program called Alpha Go and won.
- Alpha Go is a combination of reinforcement learning and Deep learning techniques which was trained on a huge database of Go matches.
- In 2017 the same time proposed Alpha Go Zero:
Learns any board Game based on the rules only by playing against version of itself. Alpha Go Zero already outperformed previous version of Alpha Go



3 hours

AlphaGo Zero plays like a human beginner, forgoing long term strategy to focus on greedily capturing as many stones as possible.



19 hours

AlphaGo Zero has learnt the fundamentals of more advanced Go strategies such as life-and-death, influence and territory.



70 hours

AlphaGo Zero plays at super-human level. The game is disciplined and involves multiple challenges across the board.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T. & Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature*, 550, 354--.

Games as Testbed for AGIs

AI is not build single purpose but receives images and general controls such as a human.

- DON on Atari Games

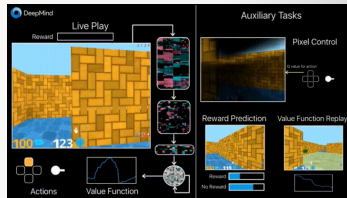
<https://deepmind.com/research/publications/playing-atari-deep-reinforcement-learning/>

- OpenAI Five (Dota2)

<https://blog.openai.com/openai-five/>

- RL for Starcraft II:

<https://deepmind.com/blog/deepmind-and-blizzard-open-starcraft-ii-ai-research-environment/>

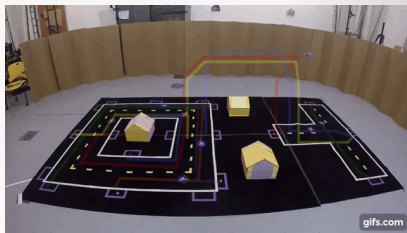


<https://techxplre.com/news/2016-11-deepmind-boost-ai-unreal-agent.html>



RL in Control and Robotics

- Control robots and drones



<https://robohub.org/drones-that-drive/>

- Chatbots

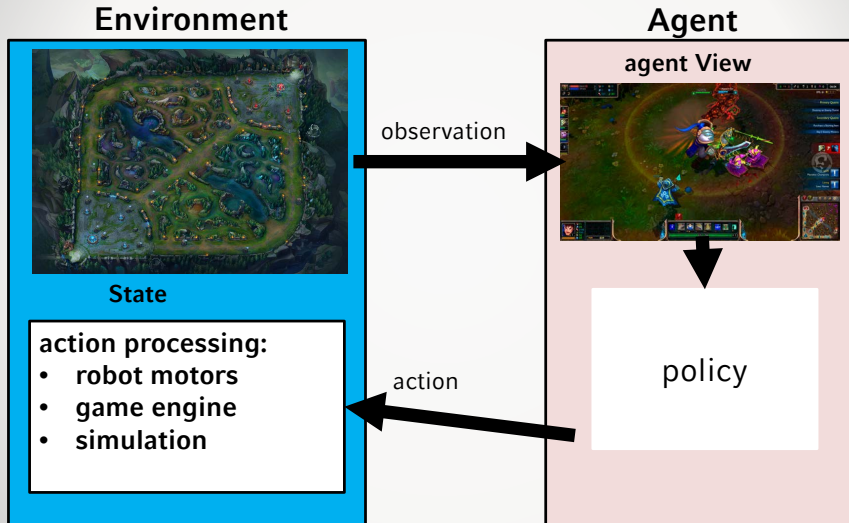
For example: Google's assistant

(<https://www.youtube.com/watch?v=D5VN56jQMWM>)

Modelfree and Modelbased Approaches

- Modelbased approaches a.k.a. planning
 - based on a complete model of the environment
 - computes rewards and actions within this models
 - optimizes policy based within this model
- Modelfree approaches
 - Assumes an interactive environment
 - Agent gets observations and selects actions
 - Environment process actions and sends new observation
 - The agents learns the best action based on the encountered interactions (episodes)

Agent and Environment



Environment

**Represents the world in which the agent is acting.
(e.g. a game, a simulation or a robot)**

- provides information about the state
(e.g. images, sounds, game states)
- receives actions and applies them to alter its state

Properties of Environments

- partially / fully observable
- with known model/ model free
- deterministic / non-deterministic
- single vs. multi-agent
- competitive vs. collaborative
- static / dynamic / semi-dynamic
- discrete / continuous (states and/or actions)

Agents

Autonomous entity within the environment.

types of agents:

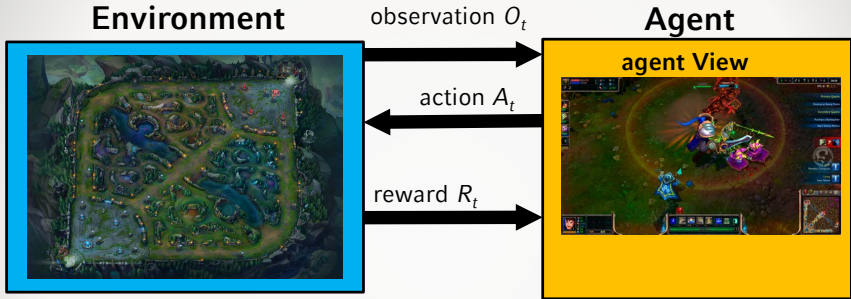
- simple reflex agent
 - => condition-action-rule
(example: If car-in-front-is-braking then initiate-braking.)
- model-based reflex agents (add internal state from history)
- goal-based agents (works towards a goal)
- **utility-based agents** (optimizes rewards/minimizes costs)
- **learning agents** (learns how to optimize rewards/costs)

Sequential Decision Making

- behavior is a sequence of actions
- sometimes immediate rewards must be sacrificed to acquire rewards in the future
example: invest today to receive a pension in old age
- short-rewards might be very unlikely in many situations
example: score a goal from your own half in football

=> intelligent behavior needs to plan ahead

Agent and Environment



At each step t the environment:

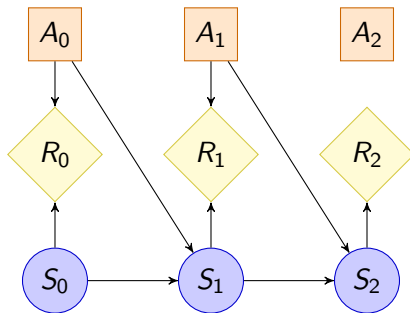
- emits observation O_t
- emits scalar reward R_t
- receives and processes action A_t

At each step t the agent:

- selects action A_t
- receives observation O_t
- receives scalar reward R_t

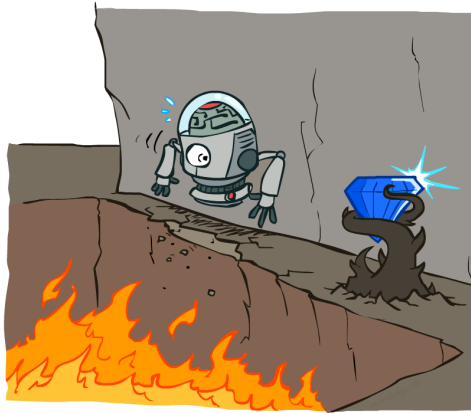
Agents

Stochastic problems



- ▶ Agent chooses action A_t at time t based on observing state S_t
- ▶ State evolves probabilistically based on current state and action taken by agent (**Markov assumption**)
- ▶ Objective is to maximize sum of rewards $R_1, \dots$

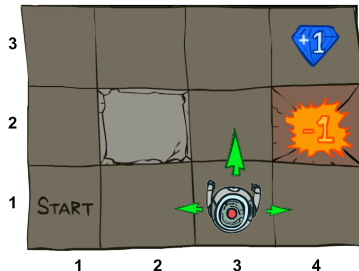
Non-Deterministic Search



Markov Decision Process

Markov Decision Processes

- An MDP is defined by:
 - A set of states $s \in S$
 - A set of actions $a \in A$
 - A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s' | s, a)$
 - Also called the model or the dynamics
 - A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
 - A start state
 - Maybe a terminal state
- MDPs are non-deterministic search problems
 - One way to solve them is with expectimax search
 - We'll have a new tool soon



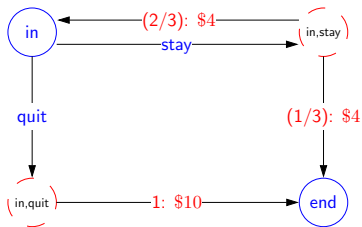
MDP for dice game



Example: dice game

For each round $r = 1, 2, \dots$

- You choose **stay** or **quit**.
- If **quit**, you get \$10 and we end the game.
- If **stay**, you get \$4 and then I roll a 6-sided dice.
 - If the dice results in 1 or 2, we end the game.
 - Otherwise, continue to the next round.



What is Markov about MDPs?

- “Markov” generally means that given the present state, the future and the past are independent
- For Markov decision processes, “Markov” means action outcomes depend only on the current state

$$\begin{aligned} P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0) \\ = \\ P(S_{t+1} = s' | S_t = s_t, A_t = a_t) \end{aligned}$$

- This is just like search, where the successor function could only depend on the current state (not the history)



Andrey Markov
(1856-1922)

Markov decision process



Definition: Markov decision process

States: the set of states

$s_{\text{start}} \in \text{States}$: starting state

Actions(s): possible actions from state s

$T(s, a, s')$: probability of s' if take action a in state s

Reward(s, a, s'): reward for the transition (s, a, s')

IsEnd(s): whether at end of game

$0 \leq \gamma \leq 1$: discount factor (default: 1)

- A **Markov decision process** has a set of states $States$, a starting state s_{start} , and the set of actions $Actions(s)$ from each state s .
- It also has a **transition distribution** T , which specifies for each state s and action a , a distribution over possible successor states s' . Specifically, we have that $\sum_{s'} T(s, a, s') = 1$ because T is a probability distribution (more on this later).
- Associated with each transition (s, a, s') is a reward, which could be either positive or negative.
- If we arrive in a state s for which $IsEnd(s)$ is true, then the game is over.
- Finally, the discount factor γ is a quantity which specifies how much we value the future and will be discussed later.

Search problems



Definition: search problem

States: the set of states

$s_{\text{start}} \in \text{States}$: starting state

$\text{Actions}(s)$: possible actions from state s

$\text{Succ}(s, a)$: where we end up if take action a in state s

$\text{Cost}(s, a)$: cost for taking action a in state s

$\text{IsEnd}(s)$: whether at end

- $\text{Succ}(s, a) \Rightarrow T(s, a, s')$
- $\text{Cost}(s, a) \Rightarrow \text{Reward}(s, a, s')$

- MDPs share many similarities with search problems, but there are differences (one main difference and one minor one).
- The main difference is the move from a deterministic successor function $\text{Succ}(s, a)$ to transition probabilities over s' . We can think of the successor function $\text{Succ}(s, a)$ as a special case of transition probabilities:

$$T(s, a, s') = \begin{cases} 1 & \text{if } s' = \text{Succ}(s, a) \\ 0 & \text{otherwise} \end{cases}.$$

- A minor difference is that we've gone from minimizing costs to maximizing rewards. The two are really equivalent: you can negate one to get the other.

Transitions



Definition: transition probabilities

The **transition probabilities** $T(s, a, s')$ specify the probability of ending up in state s' if taken action a in state s .



Example: transition probabilities

s	a	s'	$T(s, a, s')$
in	quit	end	1
in	stay	in	$2/3$
in	stay	end	$1/3$

- Just to dwell on the major difference, transition probabilities, a bit more: for each state s and action a , the transition probabilities specifies a distribution over successor states s' .

What is a solution?

Search problem: path (sequence of actions)

MDP:



Definition: policy

A **policy** π is a mapping from each state $s \in \text{States}$ to an action $a \in \text{Actions}(s)$.



Example: volcano crossing

s	$\pi(s)$
(1,1)	S
(2,1)	E
(3,1)	N
...	...

- So we now know what an MDP is. What do we do with one? For search problems, we were trying to find the minimum cost **path**.
- However, fixed paths won't suffice for MDPs, because we don't know which states the random dice rolls are going to take us.
- Therefore, we define a **policy**, which specifies an action for every single state, not just the states along a path. This way, we have all our bases covered, and know what action to take no matter where we are.
- One might wonder if we ever need to take different actions from a given state. The answer is no, since like as in a search problem, the state contains all the information that we need to act optimally for the future. In more formal speak, the transitions and rewards satisfy the **Markov property**. Every time we end up in a state, we are faced with the exact same problem and therefore should take the same optimal action.

Policies and utilities of states

- ▶ A **policy** π specifies what action to execute from every possible state
- ▶ Action to execute from state s according to π is denoted $\pi(s)$
- ▶ Expected utility of executing π when starting from s is denoted $U^\pi(s)$
- ▶ Optimal policy π^* is one that maximizes expected utility:
$$\pi^*(s) = \arg \max_{\pi} U^\pi(s)$$

Evaluating a policy



Definition: utility

Following a policy yields a **random path**.

The **utility** of a policy is the (discounted) sum of the rewards on the path (this is a random quantity).

Path	Utility
[in; stay, 4, end]	4
[in; stay, 4, in; stay, 4, in; stay, 4, end]	12
[in; stay, 4, in; stay, 4, end]	8
[in; stay, 4, in; stay, 4, in; stay, 4, in; stay, 4, end]	16
...	...



Definition: value (expected utility)

The **value** of a policy is the **expected** utility.

Evaluating a policy: volcano crossing



(or press ctrl-enter)

2.4 ↓	-0.5 ↓	-50	40
3.7 →	5 ↓	-50	31 ↑
2	12.6 →	16.3 →	26.2 ↑

$a \quad r \quad s$
 (2,1)
 E -0.1 (2,2)
 S -0.1 (3,2)
 E -0.1 (3,3)
 E -50.1 (2,3)

Value: 3.73

Utility: -36.79

- To get an intuitive feel for the relationship between a value and utility, consider the volcano example. If you press `Run` multiple times, you will get random paths shown on the right leading to different utilities. Note that there is considerable variation in what happens.
- The expectation of this utility is the **value**.
- You can run multiple simulations by increasing `numEpisodes`. If you set `numEpisodes` to 1000, then you'll see the average utility converging to the value.

Discounting



Definition: utility

Path: $s_0, a_1 r_1 s_1, a_2 r_2 s_2, \dots$ (action, reward, new state).

The **utility** with discount γ is

$$u_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

Discount $\gamma = 1$ (save for the future):

$$[\text{stay}, \text{stay}, \text{stay}, \text{stay}]: 4 + 4 + 4 + 4 = 16$$

Discount $\gamma = 0$ (live in the moment):

$$[\text{stay}, \text{stay}, \text{stay}, \text{stay}]: 4 + 0 \cdot (4 + \dots) = 4$$

Discount $\gamma = 0.5$ (balanced life):

$$[\text{stay}, \text{stay}, \text{stay}, \text{stay}]: 4 + \frac{1}{2} \cdot 4 + \frac{1}{4} \cdot 4 + \frac{1}{8} \cdot 4 = 7.5$$

- There is an additional aspect to utility: **discounting**, which captures the fact that a reward today might be worth more than the same reward tomorrow. If the discount γ is small, then we favor the present more and downweight future rewards more.
- Note that the discounting parameter is applied exponentially to future rewards, so the distant future is always going to have a fairly small contribution to the utility (unless $\gamma = 1$).
- The terminology, though standard, is slightly confusing: a larger value of the discount parameter γ actually means that the future is discounted less.

Policy evaluation



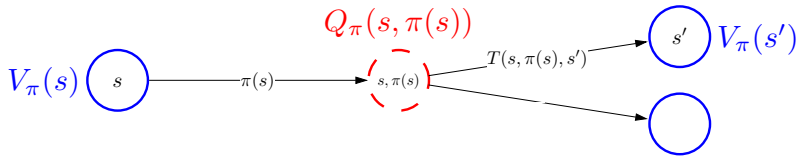
Definition: value of a policy

Let $V_{\pi}(s)$ be the expected utility received by following policy π from state s .



Definition: Q-value of a policy

Let $Q_{\pi}(s, a)$ be the expected utility of taking action a from state s , and then following policy π .



- Associated with any policy π are two important quantities, the value of the policy $V_\pi(s)$ and the Q-value of a policy $Q_\pi(s, a)$.
- In terms of the MDP graph, one can think of the value $V_\pi(s)$ as labeling the state nodes, and the Q-value $Q_\pi(s, a)$ as labeling the chance nodes.
- This label refers to the expected utility if we were to start at that node and continue the dynamics of the game.

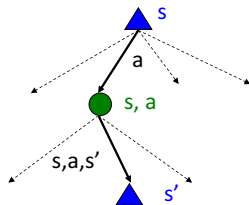
Policy iteration

- ▶ Policy iteration is one way to compute an optimal policy π^*
- ▶ The algorithm starts with any policy π_0 and iterates the following steps
 1. **Policy evaluation**: given π_i compute U^{π_i}
 2. **Policy improvement**: compute new policy from U^{π_i}
$$\pi_{i+1}(s) = \arg \max_a [R(s, a) + \gamma \sum_{s'} T(s' | s, a) U^{\pi_i}(s')]$$
- ▶ Algorithm terminates when there is no more improvement
- ▶ Since every step leads to improvement and there are finitely many policies, algorithm terminates at optimal solution

Recap: Defining MDPs

■ Markov decision processes:

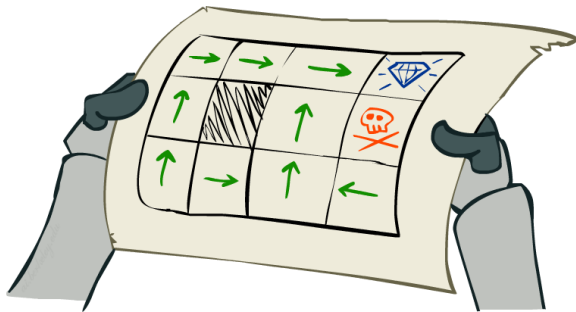
- Set of states S
- Start state s_0
- Set of actions A
- Transitions $P(s' | s, a)$ (or $T(s, a, s')$)
- Rewards $R(s, a, s')$ (and discount γ)



■ MDP quantities so far:

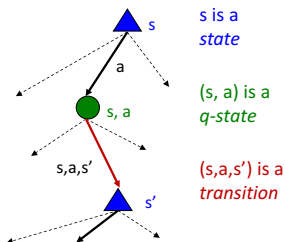
- Policy = Choice of action for each state
- Utility = sum of (discounted) rewards

Solving MDPs



Optimal Quantities

- The value (utility) of a state s :
 $V^*(s)$ = expected utility starting in s and acting optimally
- The value (utility) of a q-state (s,a) :
 $Q^*(s,a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally
- The optimal policy:
 $\pi^*(s)$ = optimal action from state s



Values of States

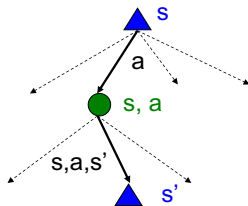
- Fundamental operation: compute the (expectimax) value of a state
 - Expected utility under optimal action
 - Average sum of (discounted) rewards
 - This is just what expectimax computed!

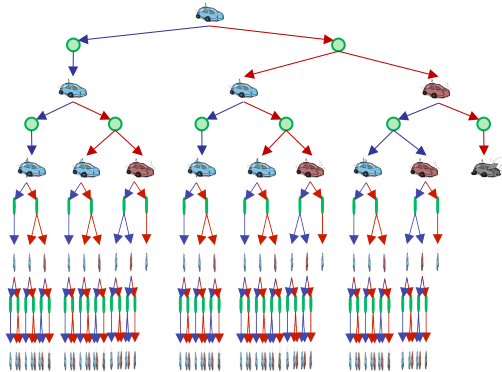
- Recursive definition of value:

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

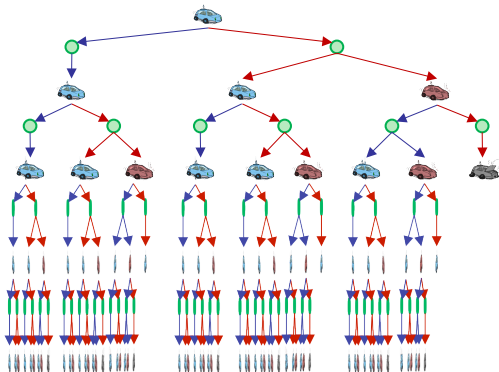
$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$





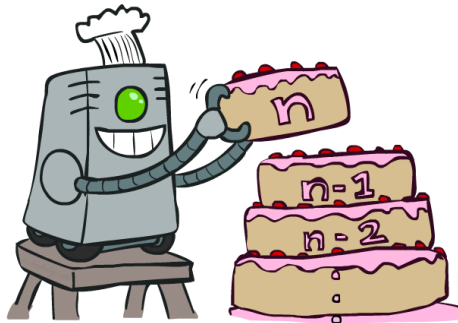
Racing Search Tree

- We're doing way too much work with expectimax!
- Problem: States are repeated
 - Idea: Only compute needed quantities once
- Problem: Tree goes on forever
 - Idea: Do a depth-limited computation, but with increasing depths until change is small
 - Note: deep parts of the tree eventually don't matter if $\gamma < 1$



Value Iteration

Value Iteration

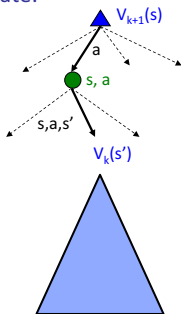


Value Iteration

- Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero
- Given vector of $V_k(s)$ values, do one ply of expectimax from each state:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

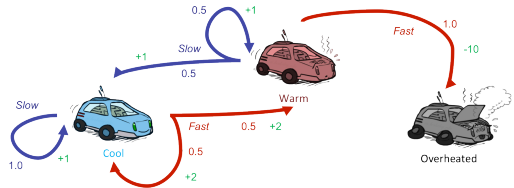
- Repeat until convergence
- Complexity of each iteration: $O(S^2A)$
- Theorem: will converge to unique optimal values
 - Basic idea: approximations get refined towards optimal values
 - Policy may converge long before values do



Example: Value Iteration

V_1

V_0

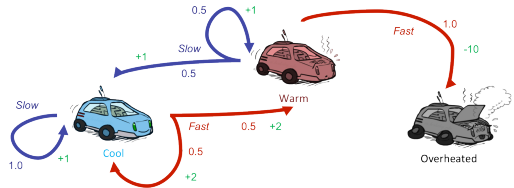


Assume no discount!

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Example: Value Iteration

			
V_2	3.5	2.5	0
V_1	2	1	0
V_0	0	0	0

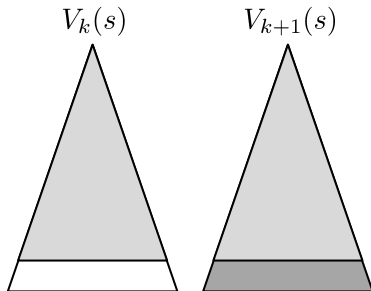


Assume no discount!

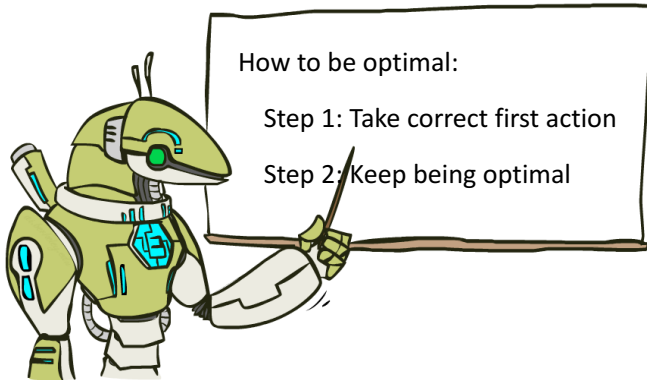
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Convergence*

- How do we know the V_k vectors are going to converge?
- Case 1: If the tree has maximum depth M , then V_M holds the actual untruncated values
- Case 2: If the discount is less than 1
 - Sketch: For any state V_k and V_{k+1} can be viewed as depth $k+1$ expectimax results in nearly identical search trees
 - The difference is that on the bottom layer, V_{k+1} has actual rewards while V_k has zeros
 - That last layer is at best all R_{MAX}
 - It is at worst R_{MIN}
 - But everything is discounted by γ^k that far out
 - So V_k and V_{k+1} are at most $\gamma^k \max |R|$ different
 - So as k increases, the values converge



The Bellman Equations



The Bellman Equations

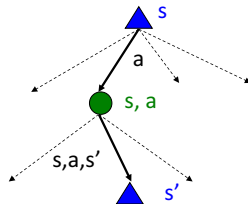
- Definition of “optimal utility” via expectimax recurrence gives a simple one-step lookahead relationship amongst optimal utility values

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- These are the Bellman equations, and they characterize optimal values in a way we'll use over and over

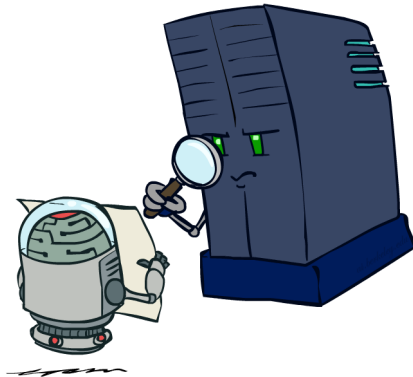


```
function VALUE-ITERATION(mdp,  $\epsilon$ ) returns a utility function
  inputs: mdp, an MDP with states  $S$ , actions  $A(s)$ , transition model  $P(s' | s, a)$ ,
           rewards  $R(s)$ , discount  $\gamma$ 
            $\epsilon$ , the maximum error allowed in the utility of any state
  local variables:  $U$ ,  $U'$ , vectors of utilities for states in  $S$ , initially zero
                      $\delta$ , the maximum change in the utility of any state in an iteration

  repeat
     $U \leftarrow U'$ ;  $\delta \leftarrow 0$ 
    for each state  $s$  in  $S$  do
       $U'[s] \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s']$ 
      if  $|U'[s] - U[s]| > \delta$  then  $\delta \leftarrow |U'[s] - U[s]|$ 
  until  $\delta < \epsilon(1 - \gamma)/\gamma$ 
  return  $U$ 
```


Policy Methods

Policy Evaluation



Policy evaluation



Key idea: iterative algorithm

Start with arbitrary policy values and repeatedly apply recurrences to converge to true values.



Algorithm: policy evaluation

Initialize $V_{\pi}^{(0)}(s) \leftarrow 0$ for all states s .

For iteration $t = 1, \dots, t_{PE}$:

For each state s :

$$V_{\pi}^{(t)}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [\underbrace{\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s')}_{Q^{(t-1)}(s, \pi(s))}]$$

- But for a much larger MDP with 100000 states, how do we efficiently compute the value of a policy?
- One option is the following: observe that the recurrences define a system of linear equations, where the variables are $V_{\pi}(s)$ for each state s and there is an equation for each state. So we could solve the system of linear equations by computing a matrix inverse. However, inverting a 100000×100000 matrix is expensive in general.
- There is an even simpler approach called **policy evaluation**. We've already seen examples of iterative algorithms in machine learning: the basic idea is to start with something crude, and refine it over time.
- Policy iteration starts with a vector of all zeros for the initial values $V_{\pi}^{(0)}$. Each iteration, we loop over all the states and apply the two recurrences that we had before. The equations look hairier because of the superscript (t) , which simply denotes the value at iteration t of the algorithm.

Policy evaluation computation

$$V_{\pi}^{(t)}(s)$$

iteration t

state s	0	-0.1	-0.2	0.7	1.1	1.6	1.9	2.2	2.4	2.6
	0	-0.1	1.8	1.8	2.2	2.4	2.7	2.8	3	3.1
	0	4	4	4	4	4	4	4	4	4
	0	-0.1	1.8	1.8	2.2	2.4	2.7	2.8	3	3.1
	0	-0.1	-0.2	0.7	1.1	1.6	1.9	2.2	2.4	2.6

- We can visualize the computation of policy evaluation on a grid, where column t denotes all the values $V_{\pi}^{(t)}(s)$ for a given iteration t . The algorithm initializes the first column with 0 and then proceeds to update each subsequent column given the previous column.
- For those who are curious, the diagram shows policy evaluation on an MDP over 5 states where state 3 is a terminal state that delivers a reward of 4, and where there is a single action, MOVE, which transitions to an adjacent state (with wrap-around) with equal probability.

Policy evaluation implementation

How many iterations (t_{PE})? Repeat until values don't change much:

$$\max_{s \in \text{States}} |V_{\pi}^{(t)}(s) - V_{\pi}^{(t-1)}(s)| \leq \epsilon$$

Don't store $V_{\pi}^{(t)}$ for each iteration t , need only last two:

$$V_{\pi}^{(t)} \text{ and } V_{\pi}^{(t-1)}$$

- Some implementation notes: a good strategy for determining how many iterations to run policy evaluation is based on how accurate the result is. Rather than set some fixed number of iterations (e.g, 100), we instead set an error tolerance (e.g., $\epsilon = 0.01$), and iterate until the maximum change between values of any state s from one iteration (t) to the previous ($t - 1$) is at most ϵ .
- The second note is that while the algorithm is stated as computing $V_{\pi}^{(t)}$ for each iteration t , we actually only need to keep track of the last two values. This is important for saving memory.

Complexity



Algorithm: policy evaluation

Initialize $V_{\pi}^{(0)}(s) \leftarrow 0$ for all states s .

For iteration $t = 1, \dots, t_{PE}$:

For each state s :

$$V_{\pi}^{(t)}(s) \leftarrow \underbrace{\sum_{s'} T(s, \pi(s), s') [\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s')]}_{Q^{(t-1)}(s, \pi(s))}$$

MDP complexity

S states

A actions per state

S' successors (number of s' with $T(s, a, s') > 0$)

Time: $O(t_{PE} S S')$

- Computing the running time of policy evaluation is straightforward: for each of the t_{PE} iterations, we need to enumerate through each of the S states, and for each one of those, loop over the successors S' . Note that we don't have a dependence on the number of actions A because we have a fixed policy $\pi(s)$ and we only need to look at the action specified by the policy.
- Advanced: Here, we have to iterate t_{PE} time steps to reach a target level of error ϵ . It turns out that t_{PE} doesn't actually have to be very large for very small errors. Specifically, the error decreases exponentially fast as we increase the number of iterations. In other words, to cut the error in half, we only have to run a constant number of more iterations.
- Advanced: For acyclic graphs (for example, the MDP for Blackjack), we just need to do one iteration (not t_{PE}) provided that we process the nodes in reverse topological order of the graph. This is the same setup as we had for dynamic programming in search problems, only the equations are different.

Policy evaluation on dice game

Let π be the "stay" policy: $\pi(\text{in}) = \text{stay}$.

$$V_{\pi}^{(t)}(\text{end}) = 0$$

$$V_{\pi}^{(t)}(\text{in}) = \frac{1}{3}(4 + V_{\pi}^{(t-1)}(\text{end})) + \frac{2}{3}(4 + V_{\pi}^{(t-1)}(\text{in}))$$

s	end	in	$(t = 100 \text{ iterations})$
$V_{\pi}^{(t)}$	0.00	12.00	

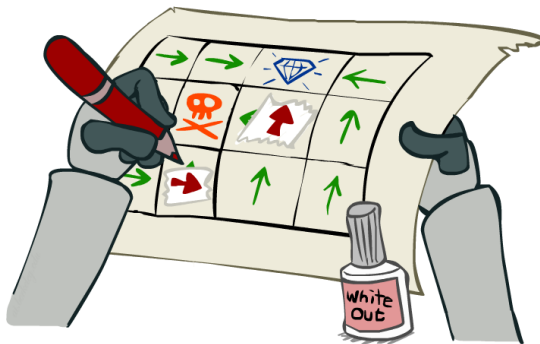
Converges to $V_{\pi}(\text{in}) = 12$.



Summary so far

- **MDP**: graph with states, chance nodes, transition probabilities, rewards
- **Policy**: mapping from state to action (solution to MDP)
- **Value of policy**: expected utility over random paths
- **Policy evaluation**: iterative algorithm to compute value of policy

Policy Iteration



Policy Iteration

- Alternative approach for optimal values:
 - **Step 1: Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities!) until convergence
 - **Step 2: Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values
 - Repeat steps until policy converges
- This is policy iteration
 - It's still optimal!
 - Can converge (much) faster under some conditions

Policy Iteration

- Evaluation: For fixed current policy π , find values with policy evaluation:

- Iterate until values converge:

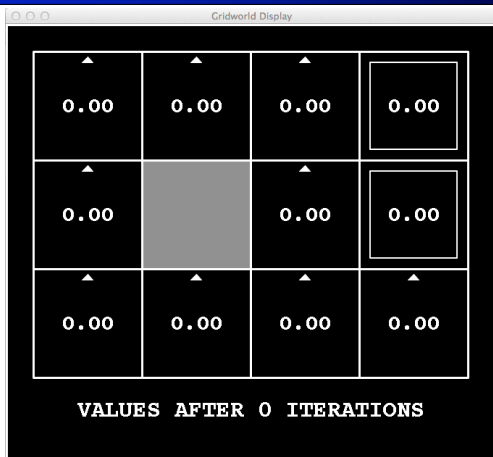
$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

- Improvement: For fixed values, get a better policy using policy extraction

- One-step look-ahead:

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

$k=0$



Noise = 0.2
Discount = 0.9
Living reward = 0

$k=1$



Noise = 0.2
Discount = 0.9
Living reward = 0

$k=2$



Noise = 0.2
Discount = 0.9
Living reward = 0

$k=3$



Noise = 0.2
Discount = 0.9
Living reward = 0

$k=4$



Noise = 0.2
Discount = 0.9
Living reward = 0

k=5



Noise = 0.2
Discount = 0.9
Living reward = 0

k=6



Noise = 0.2
Discount = 0.9
Living reward = 0

$k=7$



Noise = 0.2
Discount = 0.9
Living reward = 0

k=8



Noise = 0.2
Discount = 0.9
Living reward = 0

k=9



Noise = 0.2
Discount = 0.9
Living reward = 0

k=10



Noise = 0.2
Discount = 0.9
Living reward = 0

k=11



Noise = 0.2
Discount = 0.9
Living reward = 0

k=12



Noise = 0.2
Discount = 0.9
Living reward = 0

k=100



Noise = 0.2
Discount = 0.9
Living reward = 0

```
function POLICY-ITERATION(mdp) returns a policy
inputs: mdp, an MDP with states  $S$ , actions  $A(s)$ , transition model  $P(s' | s, a)$ 
local variables:  $U$ , a vector of utilities for states in  $S$ , initially zero
                 $\pi$ , a policy vector indexed by state, initially random

repeat
     $U \leftarrow \text{POLICY-EVALUATION}(\pi, U, \text{mdp})$ 
    unchanged?  $\leftarrow$  true
    for each state  $s$  in  $S$  do
        if  $\max_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s'] > \sum_{s'} P(s' | s, \pi[s]) U[s']$  then do
             $\pi[s] \leftarrow \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s']$ 
            unchanged?  $\leftarrow$  false
until unchanged?
return  $\pi$ 
```

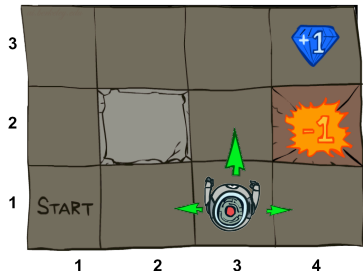
To implement the *Policy-Evaluation* step, use the following formula:

$$U_i(s) = R(s) + \gamma \sum_{s'} P(s' | s, \pi_i(s)) U_i(s').$$

Example

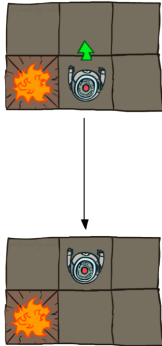
Example: Grid World

- A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
- Noisy movement: actions do not always go as planned
 - 80% of the time, the action North takes the agent North (if there is no wall there)
 - 10% of the time, North takes the agent West; 10% East
 - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
 - Small "living" reward each step (can be negative)
 - Big rewards come at the end (good or bad)
- Goal: maximize sum of rewards

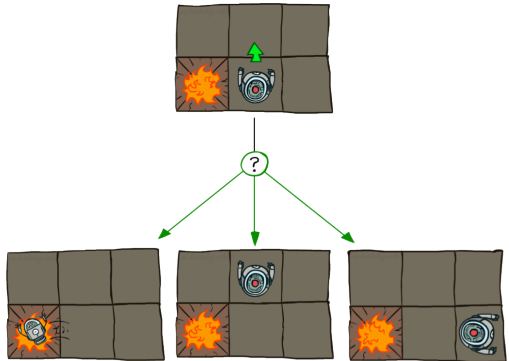


Grid World Actions

Deterministic Grid World



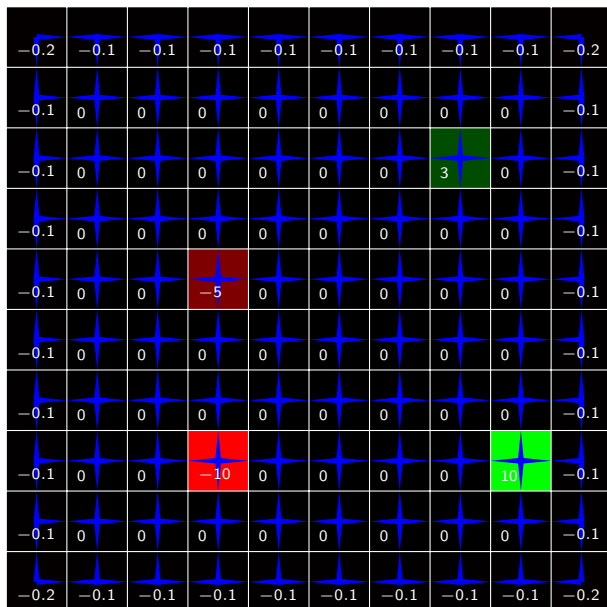
Stochastic Grid World



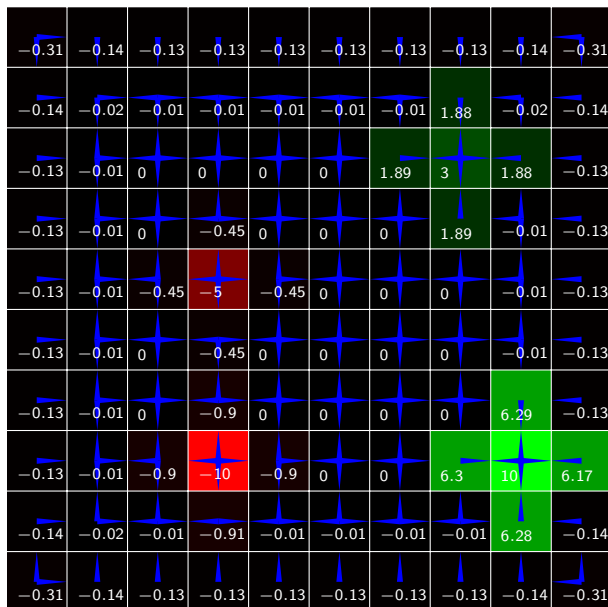
Example grid world

- ▶ **States:** cells in 10×10 grid
- ▶ **Actions:** up, down, left, and right
- ▶ **Transition model:** 0.7 chance of moving in intended direction, uniform in other directions
- ▶ **Reward:** two states have cost (and are not terminal), two have reward (and are terminal), -1 for wall crash

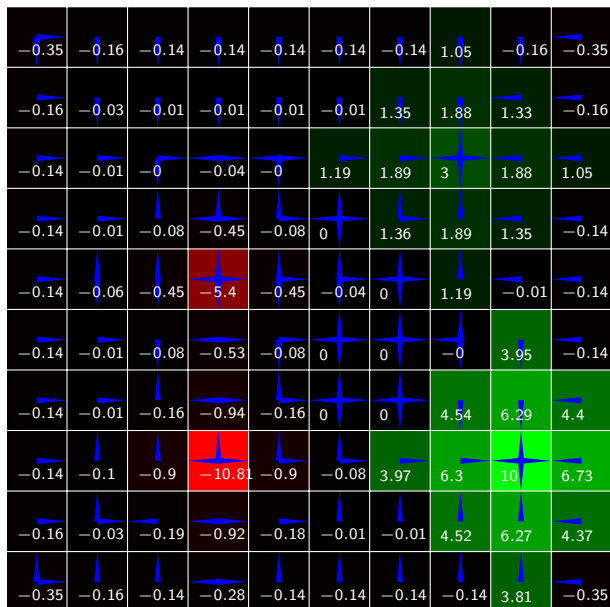
Grid world: Iteration 1 ($\gamma = 0.9$)



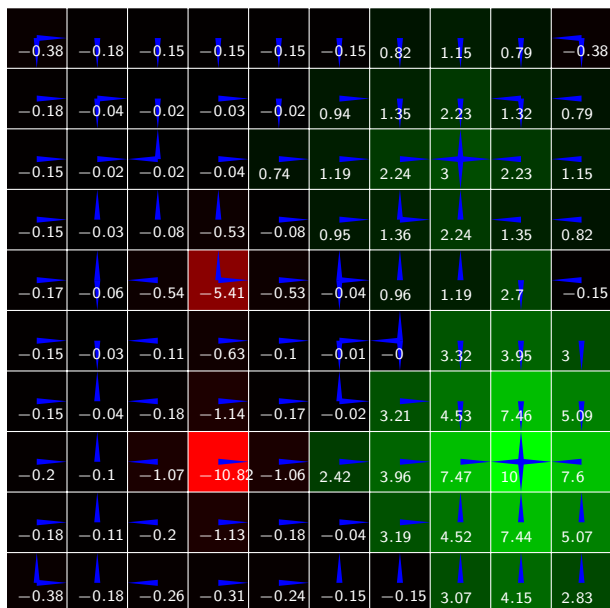
Grid world: Iteration 2 ($\gamma = 0.9$)



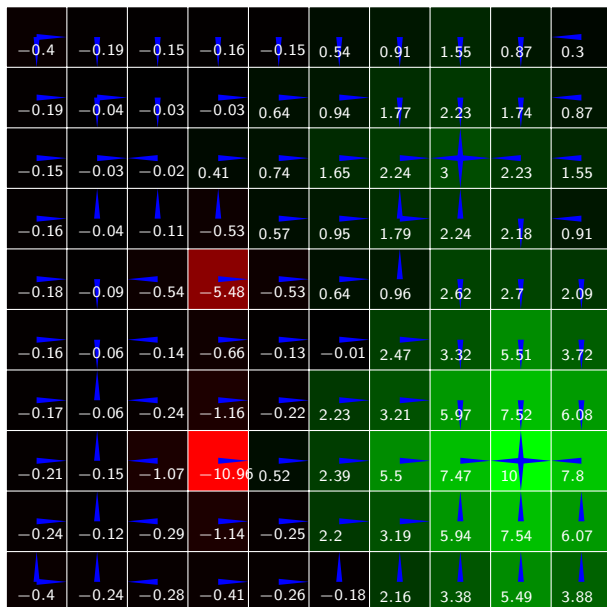
Grid world: Iteration 3 ($\gamma = 0.9$)



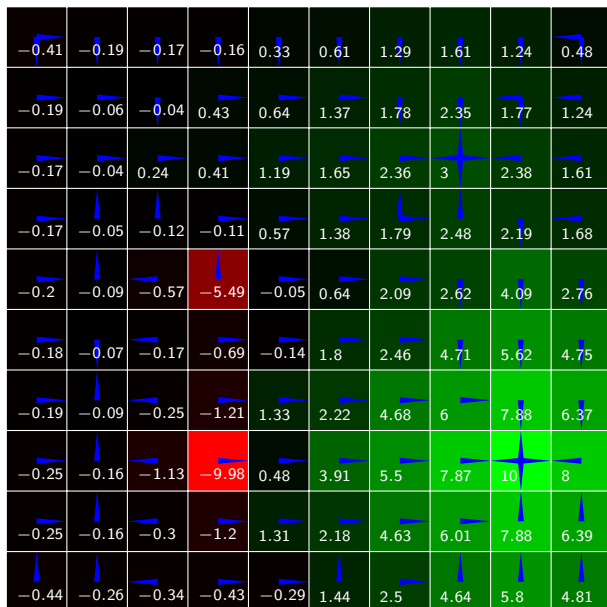
Grid world: Iteration 4 ($\gamma = 0.9$)



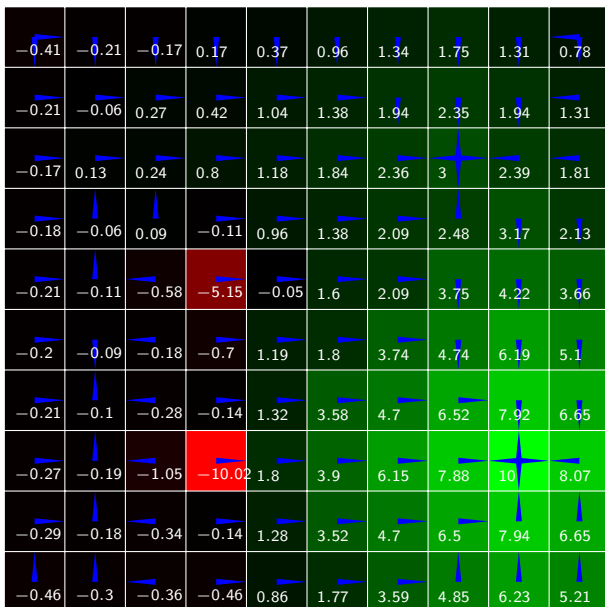
Grid world: Iteration 5 ($\gamma = 0.9$)



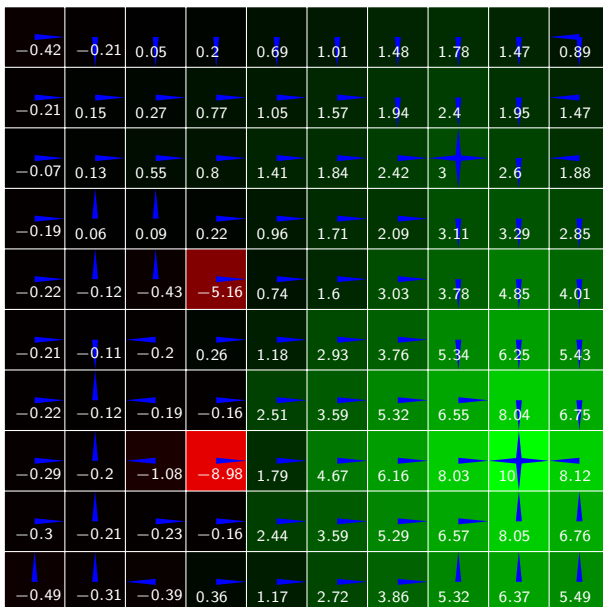
Grid world: Iteration 6 ($\gamma = 0.9$)



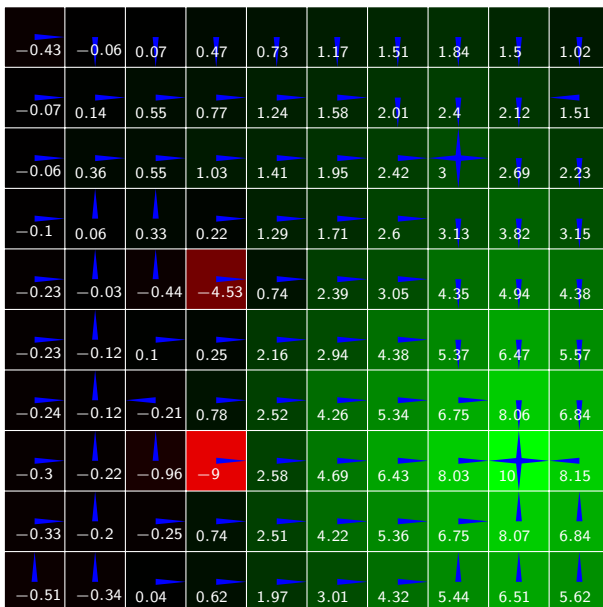
Grid world: Iteration 7 ($\gamma = 0.9$)



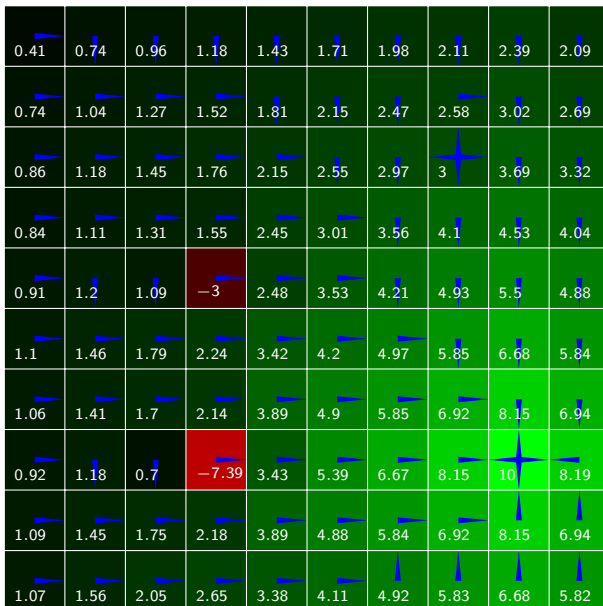
Grid world: Iteration 8 ($\gamma = 0.9$)



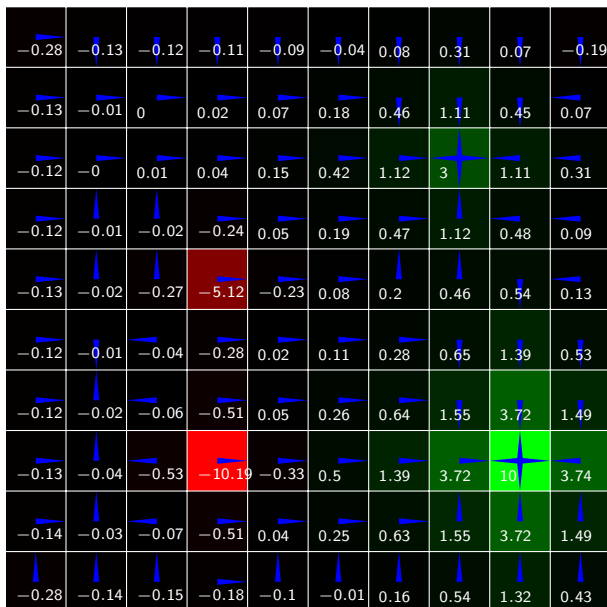
Grid world: Iteration 9 ($\gamma = 0.9$)



Grid world: Converged ($\gamma = 0.9$)



Grid world: Converged ($\gamma = 0.5$)





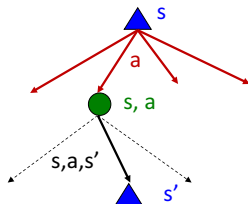
CONCLUSION

Problems with Value Iteration

- Value iteration repeats the Bellman updates:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Problem 1: It's slow – $O(S^2A)$ per iteration
- Problem 2: The “max” at each state rarely changes
- Problem 3: The policy often converges long before the values



Comparison

- Both value iteration and policy iteration compute the same thing (all optimal values)
- In value iteration:
 - Every iteration updates both the values and (implicitly) the policy
 - We don't track the policy, but taking the max over actions implicitly recomputes it
- In policy iteration:
 - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
 - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
 - The new policy will be better (or we're done)
- Both are dynamic programs for solving MDPs

Summary: MDP Algorithms

- So you want to....
 - Compute optimal values: use value iteration or policy iteration
 - Compute values for a particular policy: use policy evaluation
 - Turn your values into a policy: use policy extraction (one-step lookahead)
- These all look the same!
 - They basically are – they are all variations of Bellman updates
 - They all use one-step lookahead expectimax fragments
 - They differ only in whether we plug in a fixed policy or max over actions